

COMPACT-BENCH: ONE BUDGET AXIS FOR MEMORY COMPACTION IN LLMs AND AGENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

Methods that reduce how much of its context a language model keeps in memory are developed in four separate literatures, and each reports results in its own unit. KV-cache compression reports memory saved at fixed perplexity, prompt compression reports tokens saved at fixed accuracy, sub-quadratic architectures report recall at a fixed state size, and agent-memory systems report benchmark scores without stating how much they store. All of these methods spend the same resource, yet their numbers cannot be compared. We introduce COMPACT-Bench, which measures every method on one axis, the bytes of retained state per token of history (BPT), and asks four questions at matched budgets: how much accuracy a method keeps, whether exact or lossy retention does better, whether the system can tell what it discarded, and whether its stated confidence reflects what its memory still supports. Every self-report is checked against uncompact controls in which the fact is present or absent. On 21 settings of 16 open models from 0.5B to 24B parameters we find four results. First, the literatures disagree once every stored byte is counted. On Qwen3-1.7B a 2-bit KV cache keeps 0.98 accuracy at 22% of the full cache, where eviction, prompt compression, and summarization keep at most 0.56 within 30%, while on Qwen2.5-1.5B the same quantizer fails below 8 bits. Second, tolerance to eviction depends on model generation and not on size. In five same-family pairs from 0.5B to 14B, and across all 21 settings, every model released from November 2024 onwards loses half its accuracy at 33–69% of its budget, against 76–86% for every earlier model. Third, content-based scorers fall below random eviction only because they cannot see the question, and letting them see it recovers up to 0.98 in accuracy at the same budget. Fourth, eviction that scatters its losses leaves fragments of a fact that the model takes for the fact itself. On the models whose yes/no self-report passes both controls, SnapKV’s failures are reported as still answerable in up to 74% of cases, against at most 11% for StreamingLLM, which removes facts whole, and on the models whose stated confidence falls for an absent fact, SnapKV’s failures carry higher confidence than a fact that was never there. We release the benchmark and a reference implementation.

1 INTRODUCTION

As language models read longer inputs and run for longer, the memory that holds their context has become a major cost. The KV cache grows linearly with the number of tokens read and takes up most of the accelerator’s memory at long context (Kwon et al., 2023). An agent’s tool-call history soon exceeds any context window, and a persistent assistant accumulates state across sessions. Each of these memories has to be kept small, and a large body of work does so at every level of the stack: by evicting or quantizing cache entries (Zhang et al., 2023; Li et al., 2024; Liu et al., 2024), by pruning or gisting prompts (Jiang et al., 2023; Mu et al., 2024), by bounding the state through the design of the architecture (Gu & Dao, 2024; Bulatov et al., 2022), and by summarizing or consolidating agent memory (Packer et al., 2024; Chhikara et al., 2025).

These methods are difficult to compare with one another. Each literature holds a different quantity fixed and reports a different ratio. A practitioner choosing between a 4-bit cache, an evictor that keeps one token in eight, and a rolling summarizer has no shared number to look at, even though all three spend the same physical resource, namely bytes of retained state. Standard evaluation also

asks a narrow question. It checks whether single-turn accuracy survives compression, but it does not check whether the system can tell what it lost, whether its confidence reflects what its memory still supports, or what happens when compaction is applied again and again, as it is in any long-running agent.

COMPACT-Bench is designed to fill this gap. Its central quantity is *bytes per token of history* (BPT), the number of bytes of memory state a method keeps divided by the number of context tokens it has seen. Because retained state is eventually read through a KV cache, eviction, quantization, low-rank projection, prompt compression, and summarization can all be placed on this one axis (Section 3). The benchmark then asks four questions, each posed as a task family with fixed metrics: the *frontier* of accuracy against budget; *reversibility*, meaning whether exact or lossy retention does better at the same budget; *attribution*, meaning whether the system knows what it dropped; and the *calibration* of its confidence after compaction.

Our contributions are as follows.

- **The axis and the benchmark.** An accounting method, independent of model family, that prices every stored byte of a compaction method, including quantization metadata, together with a four-task protocol, uncompacted controls for every self-report, fixed metrics, and an open reference implementation (Section 3).
- **A small amount of theory that makes the benchmark’s questions precise.** We treat compaction as a rate–distortion problem and derive a Fano-style lower bound on error (Section 2). The bound decides which comparisons are informative. At practical budgets it is far from binding, so the failures the benchmark measures are failures of methods to keep the right bits.
- **One axis across literatures** (Section 4.1). Eviction, KV quantization, their combination, prompt compression, and summarization differ at matched bytes by up to 0.83 in accuracy, and which mechanism wins flips between two model generations of the same family.
- **Generation, not size** (Sections 4.2 and 4.3). Tolerance to eviction separates the models released before and after November 2024 at every size from 0.5B to 24B, and the failure of content-based scorers in the question-agnostic setting is a measured query-conditioning penalty of up to 0.98 in accuracy.
- **Self-report under compaction** (Section 4.5). Recording which tokens of a fact survive shows that scattered eviction leaves fragments the model mistakes for the fact, which both its yes/no self-report and its stated confidence fail to flag, while eviction that removes a fact whole is reported correctly.

Section 5 compares COMPACT-Bench with the closest benchmarks, SCBench, KVPress, and Long-MemEval (Li et al., 2025; NVIDIA, 2024; Wu et al., 2025).

2 A MINIMAL FORMALISM

Every method we benchmark does two things. It first reduces the history H , meaning everything the model could otherwise draw on, to a smaller representation Z that fits within a budget. It then answers a query using only that representation. We call the first step the *compaction operator*, $\mathcal{C}_\theta : H \mapsto Z$ with $\text{rate}(Z) \leq B$, and the second the *usage operator*, $\mathcal{U}(Z, Q) \rightarrow \hat{Y}$. What rate counts depends on the layer: cache bytes for a KV evictor, tokens for a prompt compressor, and state dimensions for an architectural model. Section 3 converts all three to one unit. Choosing $\mathcal{C}_\theta$ is then a rate–distortion problem,

$$\min_{\theta} \mathbb{E}_{(H, Q, Y)} [\ell(\mathcal{U}(\mathcal{C}_\theta(H), Q), Y)] \quad \text{s.t.} \quad \text{rate}(Z) \leq B. \quad (1)$$

Read as an information bottleneck, Eq. 1 asks $\mathcal{C}_\theta$ to maximize $I(Z; Y \mid Q)$ subject to $I(Z; H) \leq B$ (Tishby et al., 2000; Wang et al., 2025). In words, the operator should keep the parts of H that affect the answer and spend nothing on the rest.

Because $\hat{Y}$ depends on H only through Z , there is a limit on how well any method can do once the budget is too small. Proposition 1 states this limit.

Proposition 1 (Error floor). *Let $I^*(Q) = I(Y; H \mid Q)$ and let $\mathcal{Y}$ be finite. For any scheme with $\hat{Y} = \mathcal{U}(Z, Q)$,*

$$P_e \geq \frac{H(Y \mid Q) - B - 1}{\log |\mathcal{Y}|}, \quad \text{whenever } B < I^*(Q). \quad (2)$$

Appendix A derives Eq. 2 from the data-processing inequality and Fano’s inequality. The bound is a converse and holds for every θ , so once B falls below $I^*(Q)$ no choice of scorer, architecture, or retrieval method can avoid it. It also implies that the budget needed for a given accuracy grows with $I^*(Q)$, so questions that need an exact value or several facts should be hard to compress with any method. An operator that sees the query only has to keep the bits that the actual question needs, whereas one that does not see it has to cover every question that might be asked, and the difference between the two is at most $H(Q)$. The fixed-state memory limit of attention-free architectures (Wen et al., 2024) and the $\Theta(nd)$ cost of exact attention (Haris & Onak, 2025) are special cases of the same inequality. The importance scores used in practice, such as attention mass (Zhang et al., 2023; Li et al., 2024), self-information (Jiang et al., 2023), an explicit mutual-information estimate (Wang et al., 2025), an output-error bound (Feng et al., 2025), or an LLM’s own judgment of salience (Chhikara et al., 2025; Park et al., 2023), can each be seen as a cheap estimate of the true distortion $\mathcal{D}(\theta)$. No method can compute that distortion directly, because doing so would require already knowing Y .

Two properties of $\mathcal{C}_\theta$ determine which side of the floor a query falls on. **(P-rev) Reversibility** asks whether the answer to Q can still be recovered from the original history, or only from what $\mathcal{C}_\theta$ kept. The test is whether the original content can be recovered, and it is not enough for some altered version of it to remain. A quantized or merged entry stays in Z and can still be looked up, but its original value cannot be reconstructed, so it also fails reversibility, only at a finer grain than an entry that was evicted entirely. **(P-q) Query-conditioning** asks whether $\mathcal{C}_\theta$ sees Q before it decides what to drop, or whether it commits to a compaction in advance.

Together with Eq. 2, these two properties give four falsifiable predictions, one for each layer the field studies. (1) KV eviction: needle recall should collapse once the per-token budget falls below the needle’s share of attention mass, at a point determined by the entropy of where the answer is located. (2) Prompt compression: a query-aware method should beat a query-agnostic one by at most the mutual-information gap between them. (3) Fixed-state architectures: an s -bit state cannot answer a query that needs more than s bits, so state-space and linear-attention models should show a sharp recall cliff where standard attention degrades gradually. (4) Agent memory: repeated irreversible summarization should make end-task error grow faster than linearly in the number of compaction events, while a reversible, retrieval-backed memory should stay flat. Section 4 measures the recall curves that (1) concerns, a KV-cache form of the query-conditioning gap behind (2), and the equal-budget mechanism behind (4), and we leave (3) and the many-round form of (4) to future work. Attribution and calibration are new tests that ask whether a method’s report of what it dropped agrees with what it actually kept.

The floor is far from binding at the budgets people use in practice. When the 1.5B reference model has lost half its accuracy, it still retains about 48 MB of cache for a 2,000-token context, or 3.9×10^8 bits, while the answer is a six-digit number that carries at most 20 bits. Every failure we report therefore sits many orders of magnitude above Eq. 2. These are failures of a method to keep the right bits, about which a converse says nothing, and measuring them is the purpose of the benchmark. We use the bound to decide which comparisons are informative, and since $I^*(Q)$ cannot be measured we do not plug numbers into it.

3 COMPACT-BENCH

3.1 THE BPT AXIS

Four literatures compress a model’s memory, and each reports its savings in the unit its own methods produce. A KV evictor reports the fraction of tokens it keeps, a quantizer reports bits per entry, a prompt compressor reports a token ratio, and an agent-memory system reports the size of its summary store in kilobytes. None of these units can be converted into another without knowing the model’s dimensions. COMPACT-Bench replaces all four units with a single one, the bytes of

Table 1: BPT accounting: every compaction mechanism reduces to bytes retained per token of original history. b_{meta} is the per-scalar cost of a quantizer’s scales, zero points, and full-precision outliers, 1.48 bits for ours.

Mechanism	BPT cost
Full cache (uncompacted)	$2LHd \cdot \text{bytes/scalar}$
Token eviction, keep fraction f	$f \times \text{full-cache BPT}$
Quantization to b bits	$(b + b_{\text{meta}})/16 \times \text{full-cache BPT}$
Prompt or summary compression, m of n tokens kept	$(m/n) \times \text{full-cache BPT}$
Soft/gist compression, g vectors of dimension d_g	$(g d_g \cdot \text{bytes/scalar})/n$

retained memory state per token of original history (BPT), which we compute from the model’s architecture and not from whichever ratio a paper chose to report.

Once a model’s key-value dimensions are known, the conversion is mechanical. A transformer with L layers, H key-value heads, and head dimension d stores $2LHd$ scalars of KV cache per token, multiplied by the number of bytes per scalar. This uncompacted value is the full-cache BPT, and every method is measured as a fraction of it. Table 1 gives the formula for each mechanism. Everything a method stores is priced, so a quantized cache also pays for its per-group scales and zero points and for any entries it keeps at full precision. A “4-bit” cache with groups of 32 and 1% fp16 outliers therefore costs 34% of the full cache and not 25%, which is where a 66% eviction would sit (Appendix B.3).

BPT deliberately leaves compute out. It ignores the latency added by a scoring pass or a summarization call, and where we measure latency we report it next to BPT instead of combining the two into one number.

3.2 TASK FAMILIES AND METRICS

A budget axis tells a practitioner how much memory a method uses, but not whether that memory is used well. Single-turn accuracy, the number reported by almost every long-context benchmark, mixes several different questions into one score. COMPACT-Bench separates them into four task families.

The first family, *frontier*, measures how much accuracy a given budget buys. We insert a single fact at a controlled depth in a pool of natural filler text and ask a question that can only be answered from that fact. We then sweep BPT from the full-cache value down through the operating range of every method under test and report accuracy as a curve over budget, so that no single operating point is picked after the results are known. The uncompacted run sets the top of every curve.

Reversibility asks whether it matters, at the same budget, that the retained memory is exact. We compare two operators that compact only when their buffer overflows and are limited to the same token budget. The first keeps raw chunks of history and evicts the oldest chunk when full, so anything it keeps can be read back exactly. The second summarizes its buffer down to the budget on each overflow, so it covers more of the history, but none of that history can be recovered word for word. Since the two operators use the same amount of memory, any difference in accuracy between them is due to reversibility.

Attribution measures whether a system knows what it no longer has. From one compacted cache we ask two questions, a yes/no probe asking whether the system still has the fact and the real question, so that both read exactly the same retained set. The probe is scored against the correctness of the answer, and the overclaim rate is the share of wrong answers for which the probe said yes. For eviction methods we also record how much of the fact survived, as the share of layer and head slots that kept each token of its key and of its value. *Calibration* uses the confidence the system states with its answer, read from the same generation.

Neither self-report means anything unless it can report a missing fact when nothing was compacted, so both tasks include two uncompacted controls. The *present* control uses the full context with the fact in it, and the *absent* control uses the same filler with the fact removed, which no system can answer. We interpret a model’s overclaim rate only if its probe passes both controls, saying yes

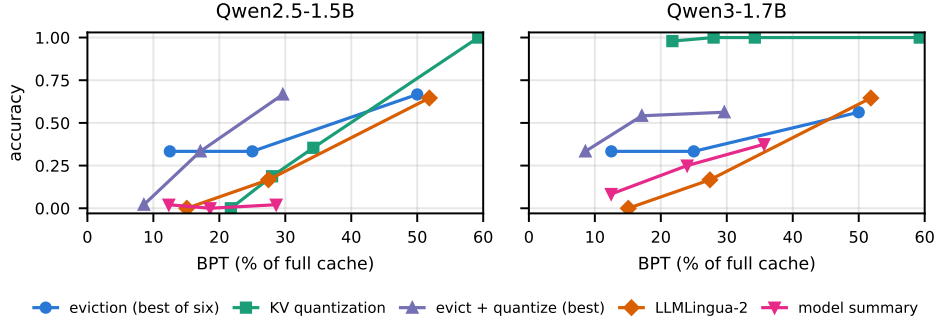


Figure 1: Needle accuracy against bytes of retained state per token of history, as a percentage of the full cache, for five compaction mechanisms answering the same 48 questions. Eviction and evict-then-quantize show the best of their methods at each budget. Quantization is priced with its metadata and outliers. On Qwen3-1.7B a 2-bit cache keeps 0.98 accuracy at 22% of the bytes, and on Qwen2.5-1.5B the same quantizer fails below 8 bits.

when the fact is present and no when it is absent, and we compare its confidence after compaction with its confidence on the absent control, which isolates what compaction adds to the instrument’s own error.

The frontier task has two variants that test its own design. In the *question-aware* variant the question is appended to the context before compression, so that the only change is what a query-scored method can see, and in the *multikey* variant three distractor facts with the same template and different keys are inserted, so that a method has to keep the right key bound to the right value. The reversibility task likewise has a *dense* variant with four times as many facts and one filler sentence per chunk.

The tasks rank methods differently. On Qwen3-4B, SnapKV answers more questions than StreamingLLM at half the budget (0.48 against 0.38), yet it claims to still have the fact on 74% of its failures, where StreamingLLM does so on none (Section 4.5). Every comparison holds everything fixed except the method and its budget, and Appendix B.2 states the protocol rules in full. Unless stated otherwise, KV methods are evaluated in the question-agnostic setting that KVPress uses by default, in which the context is compressed once before the question arrives, as when a cache is reused across queries (Appendix B.4).

4 EXPERIMENTS

The reference model is Qwen2.5-1.5B-Instruct, run on a single 8 GB consumer GPU. Section 4.1 places five mechanisms on the BPT axis for the reference model and its successor Qwen3-1.7B. Sections 4.2–4.3 study KV eviction on the reference model and then on 21 settings of 16 models, and Sections 4.4 and 4.5 cover reversibility and self-report. Appendix B lists every setting and prompt.

4.1 FIVE MECHANISMS ON ONE AXIS

Figure 1 puts eviction, KV quantization, eviction followed by quantization, LLMingua-2 prompt compression (Pan et al., 2024), and a summary written by the model itself on one axis. Each point answers the same 48 needle questions, and quantized caches pay for their scales, zero points, and fp16 outliers (Appendix B.3). Mechanisms that their literatures report in separate units turn out to differ widely at the same number of bytes. On Qwen3-1.7B, quantization keeps 1.00 accuracy at 3 bits (28% of the full cache) and 0.98 at 2 bits (22%). At about the same number of bytes, the best eviction method keeps 0.33, the best combination of eviction and quantization 0.56, the model’s own summary 0.25, and LLMingua-2 0.17.

The reference model reverses the ordering of the two KV mechanisms. Its first layer holds keys with magnitudes near 300, and even with 1% of entries kept at full precision its accuracy falls to 0.35 at 4 bits and 0.00 at 2 bits. Its best point near 30% of the bytes combines the two literatures, keeping half

the tokens with StreamingLLM and storing them in 8 bits (0.67 at 29.6%). On both models, 8-bit storage leaves the accuracy of each of the four eviction methods we combined it with unchanged to within 0.04, so adding it to an evictor removes 41% of its bytes at no cost. Summaries of this filler text rarely keep the needle, and on the reference model they answer at most 2% of questions at any budget. None of these comparisons can be read off the published numbers. “Keep 25% of tokens” and “4 bits” name the same budget only until the quantizer’s metadata is counted, and on Qwen3-1.7B the best eviction method at a quarter of the tokens answers 0.33 where the 4-bit cache answers 1.00.

4.2 THE EVICTION FRONTIER AND THE COST OF NOT SEEING THE QUESTION

Figure 3 in the appendix places six KV methods (SnapKV, StreamingLLM, TOVA, Knorm, ExpectedAttn, and a random-eviction control) on the reference model’s BPT axis over 1,395 generations, with the needle at five depths in contexts of 2,000 to 8,000 tokens. The full cache answers every question. Averaged over the five scorers, accuracy is 0.67 at 90% of the full-cache budget of 28,672 bytes per token, 0.43 at 75%, 0.20 at 50%, and zero at 10%. Three of the four content-based scorers do worse than random eviction at high budgets, and at 90% of the budget SnapKV, Knorm, and ExpectedAttn reach 0.53, 0.58, and 0.29, against 0.93 for random. The positional StreamingLLM does best at every budget and answers correctly exactly when the needle lies inside its recent window, three of five depths at half the budget (0.60) and one at a quarter (0.20).

The question-aware variant shows where the penalty comes from. SnapKV and TOVA score tokens by the attention they receive from the end of the context, which in the question-agnostic setting is filler. Once the question is appended before compression, SnapKV on the replication subset of the reference model (Appendix B) rises from 0.08 to 0.85 at half the budget and TOVA from 0.17 to 1.00, and at a quarter of the budget from 0.00 to 0.29 and from 0.04 to 0.69. Across the six models on which we ran the variant, Qwen2.5 at 0.5B, 1.5B, and 3B and Qwen3 at 0.6B, 1.7B, and 4B, SnapKV and TOVA gain a median of 0.52 in accuracy over the four largest budgets and gain on 90% of the model and budget cells. Knorm, ExpectedAttn, and random eviction do not score against the question, and they change by a median of 0.02, which makes them a built-in control. The one exception is Knorm on Qwen3-4B, whose accuracy is erratic even without the question (0.04 at 90% of the budget and 0.81 at 75%). Scorers that lose to random in the question-agnostic setting are therefore not bad scorers. They are scoring against the wrong queries, and the size of that penalty, up to 0.98 in accuracy at the same budget, is the query-conditioning gap of Section 2 measured directly. The rest of this section keeps the question-agnostic setting, which is how a cache shared across queries is compressed.

4.3 MODEL GENERATION, NOT MODEL SIZE

We ran the frontier on 21 settings of 16 models from five families (Appendix B, Table 5). The earlier group, released between August and September 2024, is Qwen2.5 at 0.5B, 1.5B, 3B, 7B, and 14B and Phi-3.5-mini. The newer group, released between November 2024 and April 2025, is Qwen3 at 0.6B, 1.7B, 4B, 8B, and 14B, OLMo-2 at 1B and 13B, SmolLM2-1.7B, Phi-4, and Mistral-Small-24B. Nine settings use 4-bit NF4 weights, either because the model does not fit one GPU at 16 bits or as a precision control, and paired runs of the same model at both precisions move the collapse point by at most 4 points. Full cache sizes range from 12,288 to 819,200 bytes per token.

Every newer model tolerates more compaction than every earlier model, at every size (Figure 2). The earlier settings lose half their accuracy at 76–86% of their own budget, and the newer ones at 33–69%. The five same-family pairs at matched

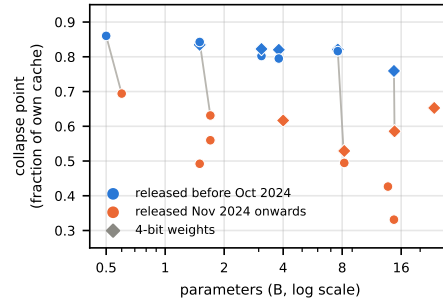


Figure 2: Budget fraction at which scorer-averaged accuracy halves, for 21 settings of 16 models. Grey lines join five same-family pairs. Every model released from November 2024 onwards (orange) tolerates more compaction than every earlier one (blue), at every size.

Table 2: Reversibility at equal token budget, compact-on-overflow (accuracy, summary / raw buffer). Standard documents hold 12 facts in about 1,600 tokens (96 questions per budget) and dense documents 48 facts in about 1,300 tokens (192 questions per budget).

Budget (% of history)	10%	25%	50%	75%	100%
Qwen2.5-1.5B, standard	0.30 / 0.07	0.47 / 0.25	0.32 / 0.45	0.40 / 0.69	1.00 / 0.94
Qwen3-1.7B, standard	0.78 / 0.07	0.98 / 0.25	1.00 / 0.50	1.00 / 0.75	1.00 / 1.00
Qwen2.5-1.5B, dense	0.11 / 0.08	0.12 / 0.22	0.27 / 0.47	0.49 / 0.71	0.90 / 0.96
Qwen3-1.7B, dense	0.19 / 0.08	0.40 / 0.22	0.70 / 0.47	0.90 / 0.72	0.93 / 0.97

size separate by 17 to 33 points: Qwen2.5-0.5B and Qwen3-0.6B collapse at 86% and 69%, the 1.5B and 1.7B models at 84% and 63%, the 3B and 4B models at 82% and 62%, the 7B and 8B models at 82% and 49%, and the two 14B models at 76% and 59%. Size has a smaller effect inside the earlier group, where Qwen2.5-14B tolerates 4 to 10 points more compaction than the smaller Qwen2.5 models, and no consistent effect inside the newer group, in which Phi-4 tolerates the most (33%) and Qwen3-0.6B the least (69%). The gap survives distractors. With three same-format needles for other keys in the context, the three Qwen2.5 models collapse at 87% or later and their Qwen3 counterparts at 68–77%.

The collapse point is a property of the model and not of its cache size in bytes. Among the nine earlier settings up to 7.6B parameters, whose full caches span a $32\times$ range, scorer-averaged accuracies at each budget fraction lie within 0.12 of one another, or within 0.16 when StreamingLLM, whose accuracy is fixed by the needle’s position, is left out. In absolute bytes the same settings disagree widely. At a quarter of its budget Phi-3.5-mini still keeps 98,304 bytes per token, more than the full cache of any Qwen2.5 model up to 7B, and answers 17% of questions. OLMo-2-13B and Phi-3.5-mini collapse at nearly the same retained state (349,365 and 312,607 bytes per token) but at 43% and 80% of their budgets.

Method rankings do not transfer between groups either. StreamingLLM is the best method at half the budget on every Qwen2.5 model, and at least three of the four content-based scorers fall below random eviction at 90% of the budget on every earlier setting. In the newer group a content-based scorer is the best method at half the budget on nine of the ten models, by up to 0.65 over StreamingLLM (SnapKV answers 96% of questions on Phi-4 at a quarter of the budget, where random eviction answers 2%), and StreamingLLM edges ahead only on SmoLM2 (0.67 against 0.65). The winning scorer changes from model to model: Knorm keeps full accuracy on Qwen3-8B down to half the budget and answers no question on OLMo-2-13B at any budget. We had predicted in advance that QK-norm, which Qwen3 adds, would explain the shift. It does not, since OLMo-2 has QK-norm and breaks Knorm while Phi-4 has none and shares its attention code with Phi-3.5-mini. The change comes from how the models were trained, and a method’s ranking has to be measured on the model it will be deployed with.

4.4 REVERSIBILITY AT EQUAL BUDGET

At a fixed token budget we compare a rolling summary, which covers the whole history but cannot be undone, with a bounded raw buffer, which covers only the most recent chunks but can be read back exactly. Both compact only when they overflow. The history is a synthetic document of 24 chunks, and at answer time the raw buffer reads back the kept chunks that mention the queried entity. Table 2 reports accuracy at five matched budgets.

The raw buffer’s accuracy tracks the fraction of the history it keeps on every model, since it keeps each fact either exactly or not at all, so it is the summary that decides which arm leads. On the reference model the two cross at 41% of the budget, and below that point the summary wins because it is the only representation that has seen everything. On standard documents the summary leads at every budget below the full history on every 16-bit model of 3B or more that we ran and on Qwen3-1.7B, Qwen3-4B, and Qwen2.5-14B, answering 0.49–1.00 at a quarter of the budget, where the raw buffer answers 0.25. These documents carry twelve facts in about 1,600 tokens of repeated filler, so a competent summarizer keeps most of them even at a tenth of the budget. Dense documents remove that slack. With 48 facts in 1,307 tokens, even the tersest encoding of every fact as an entity, attribute, and value takes 42% of the history, so a summary at a quarter of the budget can hold at

Table 3: Self-report after compaction on the six models whose probe passes both uncompacted controls, saying yes to the fact when it is present and never when it is absent (48 needles per cell). Overclaim is the share of wrong answers at compression ratio 0.5 on which the probe still claimed the fact. Confidence is the mean stated confidence on the absent control and on the wrong answers of each method. Qwen2.5-14B’s run capped generation at 50 tokens and scored a completion without a number as 0.5, so its confidence is omitted.

Model	Probe yes		Overclaim ($r=0.5$)			Stated confidence		
	present	absent	StrLLM	SnapKV	Random	absent	StrLLM	SnapKV
Qwen2.5-1.5B	0.58	0.00	0.00	0.44	0.47	0.98	0.96	0.91
Qwen2.5-3B*	0.58	0.00	0.11	0.36	0.61	0.23	0.10	0.43
Qwen2.5-7B*	0.90	0.00	0.00	0.74	0.20	0.08	0.49	0.80
Qwen2.5-14B*	0.96	0.00	0.00	0.42	0.24	—	—	—
Qwen3-0.6B	1.00	0.00	0.06	0.61	0.00	0.50	0.90	0.96
Qwen3-4B*	1.00	0.00	0.00	0.74	0.53	0.51	0.78	0.85

most about 59% of the facts and at a tenth about 24%. Qwen3-1.7B’s summary reaches 0.40 and 0.19 at those budgets, and its lead over the raw buffer at a quarter of the budget shrinks from 0.73 to 0.18. On the reference model the raw buffer now leads from a quarter of the budget upwards, and the crossover moves from 41% to 13%. Which form of retention wins at a given budget therefore depends on how much of the history the questions need, the quantity $I^*(Q)$ of Section 2, and not only on how well the model summarizes. The dense documents also repeat the generation effect. Across the three same-size pairs we ran on them (0.5B and 0.6B, 1.5B and 1.7B, 3B and 4B), the Qwen3 summary keeps more facts than the Qwen2.5 one at every budget below the full history.

4.5 WHAT A COMPACTED SYSTEM KNOWS IT LOST

Before interpreting a self-report we check that it can report a missing fact when nothing was compacted. Of the eleven models we audited, six pass both controls. The other five either claim the fact when it was never in the context (Qwen2.5-0.5B, Qwen3-1.7B, OLMo-2-1B, and SmolLM2 do so on 50–100% of the absent trials) or rarely claim it when it is present (Phi-3.5-mini, 17%), and their overclaim rates carry no information about compaction.

On the six models with a working probe, whether a system knows what it lost depends on how the eviction removes a fact (Table 3). StreamingLLM removes a contiguous span, so the needle is either kept whole or gone, and it claims the fact on at most 11% of its failures. SnapKV and random eviction scatter their losses across the context and claim it on up to 74% and 61%. The recorded survival of the needle’s tokens shows why. Pooled over these models and both ratios, the SnapKV failures on which the probe said yes had kept 70% of the key’s layer and head slots and 49% of the value’s, against 24% and 16% on the failures it correctly reported. Random eviction shows the same pattern (40% against 26%), and StreamingLLM’s 408 correctly reported failures kept none of the key. A fact that is partly present looks present to the model, and a scorer that keeps the key while dropping digits of the value produces exactly that state. Random eviction, which does no scoring at all, overclaims too, so the cause is scattered removal and not the scoring rule.

Stated confidence behaves the same way on the models where it can signal anything. On Qwen2.5-1.5B confidence is 0.98 even when the fact was never in the context, so its calibration after compaction says nothing about compaction. Five audited models do give low confidence on the absent control (0.08–0.60): Qwen2.5-3B and 7B, Qwen3-0.6B and 4B, and Phi-3.5-mini. On every one of them, confidence after a SnapKV failure is higher, by 0.16–0.72 (Table 3 shows four; Phi-3.5-mini goes from 0.60 to 0.76). After a StreamingLLM failure it is lower than after a SnapKV failure on all five. Compaction thus produces confident wrong answers in the cases where it leaves a fragment of the fact behind, and a model that can say it does not know an absent fact does not say so about a damaged one.

5 RELATED WORK

Long-context and memory benchmarks. Long-context evaluation began with the depth-by-length heatmap of Needle-in-a-Haystack (Kamradt, 2023) and has since expanded to multi-hop synthetic suites (Hsieh et al., 2024) and to broader, more realistic ones (Bai et al., 2024; Zhang et al., 2024; Yen et al., 2025). NoLiMa removes the lexical overlap between needle and query (Modarressi et al., 2025), which our needles keep for comparability with NIAH. LOCOMO and LongMemEval evaluate multi-session dialogue and memory abilities at the agent level (Maharana et al., 2024; Wu et al., 2025). Two suites are closest to compaction itself. SCBench studies KV-centric long-context methods across generation, compression, retrieval, and loading (Li et al., 2025), and KVPress provides a common harness for eviction and quantization methods (NVIDIA, 2024). Neither reports results on a shared budget axis or tests whether a system knows what it discarded. COMPACT-Bench builds directly on both. It reuses their infrastructure and adds the BPT axis, the reversibility, attribution, and calibration task families, and the uncompacted controls.

Compaction methods. KV-cache methods evict scored tokens (Zhang et al., 2023; Li et al., 2024; Xiao et al., 2024; Oren et al., 2024) or lower the bit-width of every entry (Liu et al., 2024; Hooper et al., 2025). Prompt compression deletes low-value tokens (Jiang et al., 2023; Pan et al., 2024) or maps the context to trained latent vectors (Mu et al., 2024). Architectural methods build a fixed-size memory into the model (Gu & Dao, 2024; Bulatov et al., 2022), and agent-memory systems page content in and out of an external store (Packer et al., 2024) or extract and update facts (Chhikara et al., 2025).

Calibration. Work on stated confidence has studied uncompressed models, which evaluate their own answers well in a suitable format (Kadavath et al., 2022) and state confidence in words that is often better calibrated than token probabilities (Tian et al., 2023; Xiong et al., 2023). It does not ask what happens once the context has been compacted. Our absent-fact control shows that on several small models verbal confidence cannot report a missing fact even without compaction, so a calibration result under compaction has to be read against that control.

6 LIMITATIONS

Our largest model has 24B parameters, and we evaluate open-weight models only, since the benchmark needs direct access to the KV cache. KVPress cannot compress a model sharded across GPUs, so the largest models run with 4-bit weights, which in paired runs moves the collapse point by at most 4 points. The generation split rests on ten newer and six earlier models from five families; it separates the groups at every size we tested but does not identify which training choice caused it, and our QK-norm hypothesis for it failed. The cross-mechanism comparison covers two models, and the question-aware and multikey variants six settings. The self-report findings rest on the six models whose probe passes both uncompacted controls, since on the other five the probe cannot report a missing fact at all. Prediction (3) and the many-round form of (4) are left to future work, since testing (3) requires state-space models, which KVPress does not yet support. All tasks use synthetic English text.

7 CONCLUSION

COMPACT-Bench places memory-compaction methods from four literatures on a single axis, bytes of retained state per token of history, and measures what single-turn accuracy leaves out. Counted in bytes, the literatures disagree by far more than their own units suggest, and the winning mechanism changes with the model. Tolerance to eviction is set by model generation and not by size, and content-based scorers that lose to random eviction do so because they cannot see the question. Eviction that scatters its losses leaves fragments of a fact that the model takes for the fact itself, so it claims knowledge it has lost, while eviction that removes facts whole is reported correctly. We release COMPACT-Bench so that every compaction method can report, beside its accuracy at each budget, whether the system can say what it lost.

AI USE STATEMENT

We used generative AI tools (Anthropic’s Claude, through the Claude Code agent) for tasks with required disclosure: helping develop the rate–distortion framing, formulating the error floor and drafting its proof (Proposition 1, Appendix A), proposing hypotheses, including the QK-norm hypothesis of Section 4.3, designing the experiments, including the cross-scale ladders, precision controls, and uncompact self-report controls, implementing the benchmark harness, which also generates the synthetic evaluation documents, and interpreting results. We did not use generative AI tools for translation, and cleaning existing datasets and qualitative data analysis are not applicable to this work. We also used them for tasks with recommended disclosure: writing and editing code, creating figures, drafting and editing the paper, and searching the related literature. We have reviewed all AI-assisted work. Every number reported in the paper was recomputed from the per-generation records released with the benchmark, each change to the harness was tested against the reference implementation it modifies, and the hypothesis stated in advance in Section 4.3 was committed to version control before the runs that tested it. We take responsibility for the final content of this work, including text, claims or artifacts produced with the aid of generative AI.

REPRODUCIBILITY STATEMENT

Appendix B lists the hardware, model, and weight precision of every setting (Tables 4 and 5), the BPT accounting (Appendix B.3), and every prompt, sweep, and metric (Appendix B.4), including the correction to the reversibility harness and the handling of completions without a stated confidence. Appendix A states the assumptions of Proposition 1 and gives its full derivation. The supplementary material contains an anonymized copy of the benchmark code and the per-generation JSON records behind every reported number.

REFERENCES

- Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. Longbench: A bilingual, multitask benchmark for long context understanding, 2024. URL <https://arxiv.org/abs/2308.14508>.
- Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer, 2022. URL <https://arxiv.org/abs/2207.06881>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Yuan Feng, Junlin Lv, Yukun Cao, Xike Xie, and S. Kevin Zhou. Ada-kv: Optimizing kv cache eviction by adaptive budget allocation for efficient llm inference, 2025. URL <https://arxiv.org/abs/2407.11550>.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2024. URL <https://arxiv.org/abs/2312.00752>.
- Themistoklis Haris and Krzysztof Onak. Compression barriers for autoregressive transformers, 2025. URL <https://arxiv.org/abs/2502.15955>.
- Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W. Mahoney, Yakun Sophia Shao, Kurt Keutzer, and Amir Gholami. Kvquant: Towards 10 million context length llm inference with kv cache quantization, 2025. URL <https://arxiv.org/abs/2401.18079>.
- Cheng-Ping Hsieh, Simeng Sun, Samuel Krizan, Shantanu Acharya, Dima Rekeshe, Fei Jia, Yang Zhang, and Boris Ginsburg. Ruler: What’s the real context size of your long-context language models?, 2024. URL <https://arxiv.org/abs/2404.06654>.
- Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmllingua: Compressing prompts for accelerated inference of large language models, 2023. URL <https://arxiv.org/abs/2310.05736>.

- Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, Scott Johnston, Sheer El-Showk, Andy Jones, Nelson Elhage, Tristan Hume, Anna Chen, Yuntao Bai, Sam Bowman, Stanislav Fort, Deep Ganguli, Danny Hernandez, Josh Jacobson, Jackson Kernion, Shauna Kravec, Liane Lovitt, Kamal Ndousse, Catherine Olsson, Sam Ringer, Dario Amodei, Tom Brown, Jack Clark, Nicholas Joseph, Ben Mann, Sam McCandlish, Chris Olah, and Jared Kaplan. Language models (mostly) know what they know, 2022. URL <https://arxiv.org/abs/2207.05221>.
- Greg Kamradt. Needle in a haystack – pressure testing LLMs. https://github.com/gkamradt/LLMTest_NeedleInAHaystack, 2023. Software.
- Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention, 2023. URL <https://arxiv.org/abs/2309.06180>.
- Yucheng Li, Huiqiang Jiang, Qianhui Wu, Xufang Luo, Surin Ahn, Chengruidong Zhang, Amir H. Abdi, Dongsheng Li, Jianfeng Gao, Yuqing Yang, and Lili Qiu. Scbench: A kv cache-centric analysis of long-context methods, 2025. URL <https://arxiv.org/abs/2412.10319>.
- Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. Snapkv: Llm knows what you are looking for before generation, 2024. URL <https://arxiv.org/abs/2404.14469>.
- Zirui Liu, Jiayi Yuan, Hongye Jin, Shaochen Zhong, Zhaozhuo Xu, Vladimir Braverman, Beidi Chen, and Xia Hu. Kivi: A tuning-free asymmetric 2bit quantization for kv cache, 2024. URL <https://arxiv.org/abs/2402.02750>.
- Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents, 2024. URL <https://arxiv.org/abs/2402.17753>.
- Ali Modarressi, Hanieh Deilamsalehy, Franck Dernoncourt, Trung Bui, Ryan A. Rossi, Seunghyun Yoon, and Hinrich Schütze. Nolina: Long-context evaluation beyond literal matching, 2025. URL <https://arxiv.org/abs/2502.05167>.
- Jesse Mu, Xiang Lisa Li, and Noah Goodman. Learning to compress prompts with gist tokens, 2024. URL <https://arxiv.org/abs/2304.08467>.
- NVIDIA. KVPress: LLM KV cache compression made easy. <https://github.com/NVIDIA/kvpress>, 2024. Software library.
- Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state rnns, 2024. URL <https://arxiv.org/abs/2401.06104>.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. Memgpt: Towards llms as operating systems, 2024. URL <https://arxiv.org/abs/2310.08560>.
- Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. Llm2lingua-2: Data distillation for efficient and faithful task-agnostic prompt compression, 2024. URL <https://arxiv.org/abs/2403.12968>.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior, 2023. URL <https://arxiv.org/abs/2304.03442>.
- Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher D. Manning. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models fine-tuned with human feedback, 2023. URL <https://arxiv.org/abs/2305.14975>.

- Naftali Tishby, Fernando C. Pereira, and William Bialek. The information bottleneck method, 2000. URL <https://arxiv.org/abs/physics/0004057>.
- Yihang Wang, Xu Huang, Bowen Tian, Yueyang Su, Lei Yu, Huaming Liao, Yixing Fan, Jiafeng Guo, and Xueqi Cheng. Quito-x: A new perspective on context compression from the information bottleneck theory, 2025. URL <https://arxiv.org/abs/2408.10497>.
- Kaiyue Wen, Xingyu Dang, and Kaifeng Lyu. Rnns are not transformers (yet): The key bottleneck on in-context retrieval, 2024. URL <https://arxiv.org/abs/2402.18510>.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory, 2025. URL <https://arxiv.org/abs/2410.10813>.
- Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2024. URL <https://arxiv.org/abs/2309.17453>.
- Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can llms express their uncertainty? an empirical evaluation of confidence elicitation in llms, 2023. URL <https://arxiv.org/abs/2306.13063>.
- Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. Helmet: How to evaluate long-context language models effectively and thoroughly, 2025. URL <https://arxiv.org/abs/2410.02694>.
- Xinrong Zhang, Yingfa Chen, Shengding Hu, Zihang Xu, Junhao Chen, Moo Khai Hao, Xu Han, Zhen Leng Thai, Shuo Wang, Zhiyuan Liu, and Maosong Sun. ∞ bench: Extending long context evaluation beyond 100k tokens, 2024. URL <https://arxiv.org/abs/2402.13718>.
- Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H₂O: Heavy-hitter oracle for efficient generative inference of large language models, 2023. URL <https://arxiv.org/abs/2306.14048>.

A DERIVATION OF THE ERROR FLOOR

We restate Proposition 1 with its assumptions made explicit and give the short derivation. Fix a finite answer set $\mathcal{Y}$, use base-two logarithms, and let the budget bound the description length of the representation, $H(Z) \leq B$ bits. Any physical memory budget gives such a bound. Assume the usage operator reads the history only through the compact representation and the query, so $\hat{Y} = \mathcal{U}(Z, Q)$, and Y and $\hat{Y}$ are conditionally independent given (Z, Q) . All quantities below are implicitly conditioned on Q and averaged over its distribution, and $P_e = \Pr[\hat{Y} \neq Y]$.

Conditioned on Q , the representation $Z = \mathcal{C}(H, Q)$ is a function of H alone, whether or not $\mathcal{C}$ also depends on Q , so $Y - H - Z$ forms a Markov chain given Q . The same holds for $Y - Z - \hat{Y}$, since $\hat{Y}$ is a function of (Z, Q) . Applying the data-processing inequality along each chain gives two separate bounds on how much $\hat{Y}$ can tell us about Y . The first comes from the chain through H :

$$I(Y; \hat{Y} | Q) \leq I(Y; Z | Q) \leq I(Y; H | Q) = I^*(Q). \quad (3)$$

This says that $\hat{Y}$ cannot carry more information about Y than the representation it was built from, and that the representation cannot carry more than the raw history. The second bound comes from the budget, since $\hat{Y}$ is a deterministic function of (Z, Q) :

$$I(Y; \hat{Y} | Q) \leq H(\hat{Y} | Q) \leq H(Z | Q) \leq H(Z) \leq B. \quad (4)$$

Together these give $I(Y; \hat{Y} | Q) \leq \min(I^*(Q), B)$. In other words, no scheme can pass more information about the answer than the smaller of what the history contains and what the budget can hold. Fano’s inequality turns this information bound into an error bound. Using the standard relaxations $H_b(P_e) \leq 1$ and $\log(|\mathcal{Y}| - 1) \leq \log |\mathcal{Y}|$, Fano gives $H(Y | \hat{Y}, Q) \leq 1 + P_e \log |\mathcal{Y}|$.

Since $H(Y | \hat{Y}, Q) = H(Y | Q) - I(Y; \hat{Y} | Q)$ by the definition of conditional entropy, rearranging and substituting the information bound above gives

$$P_e \geq \frac{H(Y | Q) - I(Y; \hat{Y} | Q) - 1}{\log |\mathcal{Y}|} \geq \frac{H(Y | Q) - \min(I^*(Q), B) - 1}{\log |\mathcal{Y}|}. \quad (5)$$

When the budget is the limiting factor, $B < I^*(Q)$, the minimum equals B , which gives Eq. 2.

Three points matter for how the bound should be read. First, it is a converse and not an achievability result. It lower-bounds the error of every scheme that satisfies the assumptions above, but it says nothing about whether a particular scheme comes close to it, so a well-designed method may sit well above the floor. Second, it is loose, because the relaxations used in the Fano step are crude. It is informative only when $H(Y | Q)$ exceeds the budget by a clear margin, which is the regime it is meant to describe. Third, query-conditioning does not move the $I^*(Q)$ ceiling, since that quantity is defined without reference to any particular operator and the derivation never assumed that $\mathcal{C}$ ignores the query. What query-conditioning changes is how efficiently a method spends its budget on the way to that ceiling. The penalty for ignoring the query, discussed in Section 2, is the gap between the conditional and unconditional rates a representation would need, $I(Z; H | Q) - I(Z; H)$, which is at most $H(Q)$ and usually much smaller in practice.

B BENCHMARK DETAILS

B.1 HARDWARE AND MODELS

The reference-model frontier (Figure 3) uses Qwen2.5-1.5B-Instruct on a single consumer GPU with 8 GB of VRAM (RTX 4060), with peak usage of 3.6 GiB in fp16 on the 8,000-token contexts. The standard-document rows of Table 2 for Qwen2.5-1.5B come from the corrected-harness run on the server described below. We chose a small reference scale so that the protocol can be reproduced on commodity hardware. The harness accepts any Hugging Face causal LM via `--model`, and the accounting in Appendix B.3 does not depend on the model. The first cross-scale ladder repeats a subset of the protocol on the same GPU at the five further settings listed in Table 4. Every setting fits within 6.4 GiB because weights above 1.5B are loaded in 4-bit NF4 (`--quant nf4`). This gives up some weight precision in exchange for testing three to four times as many parameters on the same card.

Table 4: Cross-scale hardware settings, measured on the RTX 4060.

Model	Precision	Full-cache BPT	Peak VRAM
Qwen2.5-0.5B-Instruct	fp16	12,288 B/tok	1.12 GiB
Qwen2.5-1.5B-Instruct	fp16	28,672 B/tok	3.25 GiB
Qwen2.5-1.5B-Instruct	nf4	28,672 B/tok	2.69 GiB
Qwen2.5-3B-Instruct	nf4	36,864 B/tok	5.19 GiB
Phi-3.5-mini-instruct	nf4	393,216 B/tok	6.39 GiB
Qwen2.5-7B-Instruct	nf4	57,344 B/tok	5.94 GiB

The second ladder ran on a server with two 32 GB RTX 5000 Ada GPUs, one job per GPU. It adds 16-bit runs of Qwen2.5-3B, Qwen2.5-7B, and Phi-3.5-mini and the five newer models, with peak VRAM of 16.2 GiB for Qwen3-8B, 28.9 GiB for Phi-4, and 29.2 GiB for OLMo-2-13B in bf16. KVPress cannot compress a model that accelerate shards across GPUs, because the attention mask is built for the compressed length while layers on the other device keep the full cache. Qwen3-14B and Mistral-Small-24B, which do not fit on one card at 16 bits, therefore run in NF4. KVPress also rebuilds queries for scoring without OLMo-2’s QK-norm, and multiplies OLMo-2’s float32 rotary tables against bf16 queries. Our harness corrects both inside the scoring hook, which leaves the model’s own forward pass and the cache it compresses unchanged.

A third batch adds the models that separate generation from size. Qwen3 at 0.6B, 1.7B, and 4B (the last in NF4), SmolLM2-1.7B, and OLMo-2-1B ran on the RTX 4060, as did every mechanism, question-aware, multikey, and dense reversibility run and every audit except Qwen2.5-14B’s. Qwen2.5-14B ran on the server from a published bitsandbytes NF4 checkpoint

(unsloth/Qwen2.5-14B-Instruct-bnb-4bit), the same quantization our loader applies to Qwen3-14B and Mistral-Small-24B, because the server’s scratch disk could not hold its 16-bit weights.

Table 5: Cross-scale results for each setting (replication subset: lengths {2000, 4000}, depths {0.1, 0.5, 0.9}, 8 trials, 48 generations per cell). Collapse is the interpolated budget fraction at which accuracy averaged over the five non-random scorers first falls below half the model’s own baseline, on the grid {0.9, 0.75, 0.5, 0.25, 0.1}; the content-only column leaves out StreamingLLM, and >90% means accuracy is already below half at the largest budget. Multikey is the collapse point with three distractor needles (– where not run). Crossover is where the raw buffer overtakes the summary in the standard reversibility task (– means the summary is at least as accurate at every budget below the full history, † run with the earlier summary cap of Appendix B.4, n/r not run with the corrected cap). SmolLM2’s summaries copy the repeated filler verbatim and keep no facts, which accounts for its early crossover. The rule separates models released before October 2024 (top) from later ones (bottom).

Model (precision)	Released	Full BPT	Collapse	Content only	Multikey	Crossover
Qwen2.5-0.5B (fp16)	2024-09	12,288	86%	>90%	>90%	48%
Qwen2.5-1.5B (fp16)	2024-09	28,672	84%	89%	88%	41%
Qwen2.5-1.5B (nf4)	2024-09	28,672	83%	88%	–	n/r
Qwen2.5-3B (fp16)	2024-09	36,864	80%	87%	–	–
Qwen2.5-3B (nf4)	2024-09	36,864	82%	88%	87%	n/r
Qwen2.5-7B (fp16)	2024-09	57,344	82%	90%	–	–
Qwen2.5-7B (nf4)	2024-09	57,344	82%	>90%	–	n/r
Qwen2.5-14B (nf4)	2024-09	196,608	76%	78%	–	–
Phi-3.5-mini (bf16)	2024-08	393,216	80%	87%	–	–
Phi-3.5-mini (nf4)	2024-08	393,216	82%	90%	–	n/r
Qwen3-0.6B (bf16)	2025-04	114,688	69%	71%	77%	61%
OLMo-2-1B (bf16)	2025-04	131,072	49%	53%	–	40%
Qwen3-1.7B (bf16)	2025-04	114,688	63%	63%	69%	–
SmolLM2-1.7B (bf16)	2024-11	196,608	56%	61%	–	14%
Qwen3-4B (nf4)	2025-04	147,456	62%	62%	68%	–
Qwen3-8B (bf16)	2025-04	147,456	49%	46%	–	–†
Qwen3-8B (nf4)	2025-04	147,456	53%	50%	–	n/r
OLMo-2-13B (bf16)	2024-11	819,200	43%	44%	–	n/r
Phi-4 (bf16)	2024-12	204,800	33%	31%	–	–†
Qwen3-14B (nf4)	2025-04	163,840	59%	58%	–	n/r
Mistral-Small-24B (nf4)	2025-01	163,840	65%	71%	–	n/r

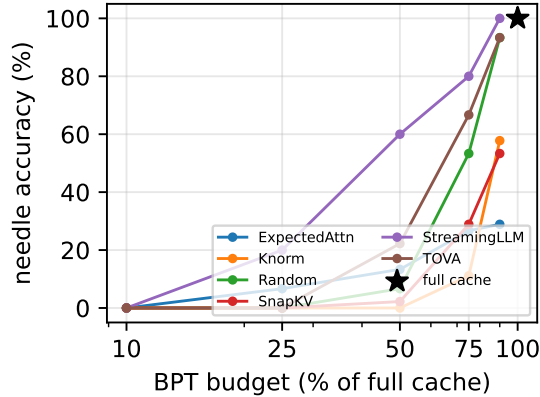


Figure 3: Needle accuracy against BPT for six KV methods on the reference model Qwen2.5-1.5B (1,395 generations), with the cache compressed before the question arrives (Section 4.2).

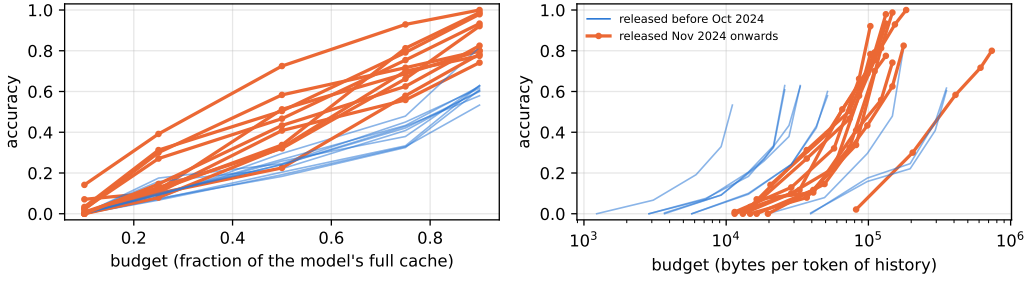


Figure 4: The sweep of Table 5 plotted against budget fraction (left) and against absolute bytes per token (right, log scale), for all 21 settings. In bytes the curves spread over two orders of magnitude, while as a fraction they separate into models released before October 2024 and those released from November 2024 onwards.

B.2 PROTOCOL RULES

Every comparison in COMPACT-Bench holds everything fixed except the method under test and its budget. The same random seed generates the same needle documents and fact sets at every point on a sweep, so two methods compared at the same budget answer the same questions and not different questions drawn from a shared distribution. This matters because a method’s difficulty depends heavily on where the answer sits in the document, and letting that position vary between conditions would confound the comparison. We report the number of generations behind every summary statistic, because our cells are small (48 generations in most tasks), and a mean given without its sample size is easy to over-read. Finally, no method chooses the budget at which it is evaluated. Every method in a sweep runs at the same fixed set of budgets, so no method can be shown only at its most favourable operating point, as often happens when individual method papers report a single ratio. Appendix B.4 describes the question-agnostic setting in which all KV methods are run.

B.3 BPT ACCOUNTING, WORKED EXAMPLE

For a model with L layers, H KV heads, and head dimension d , one full-precision token of KV cache costs $2LHd \cdot (\text{bytes/scalar})$ bytes, where the factor of 2 accounts for keys and values. For Qwen2.5-1.5B-Instruct ($L=28$, $H=2$, $d=128$, fp16), this is $2 \cdot 28 \cdot 2 \cdot 128 \cdot 2 = 28,672$ bytes/token, the full-cache value in Figure 3. An evictor that keeps a fraction f of entries costs $f \times 28,672$ bytes/token. A b -bit quantizer stores b bits per scalar, and it also stores metadata that the “ b bits” of its literature leaves out. Our quantizer (Section 4.1) groups keys per channel and values per token in groups of 32 with one fp16 scale and zero point per group, which adds 1 bit per scalar, and keeps the largest 1% of entries in each layer at fp16 with a 32-bit index, as KVQuant does (Hooper et al., 2025), which adds 0.48 bits. It therefore costs $(b + 1.48)/16$ of the full cache, 34.2% at 4 bits and 21.8% at 2 bits. Without the outlier set, 4-bit keys break Qwen2.5 entirely, because its first layer holds key channels with magnitudes near 300. Reported in their own units, 75% eviction and 4-bit quantization look equivalent (“25% of tokens kept” and “4 bits of 16”), but once every stored byte is counted the 4-bit cache costs as much as a 66% eviction. Eviction followed by quantization multiplies the two costs, so keeping half the tokens at 8 bits costs $0.5 \times 9.48/16 = 29.6\%$. Prompt compression from n history tokens to m kept tokens costs $(m/n) \times 28,672$ bytes/token, and soft or gist compression to g vectors of dimension d_g costs $g \cdot d_g \cdot (\text{bytes/scalar})/n$ bytes/token, amortized in the same way. The same short accounting module prices all four mechanisms (`compactbench/bpt.py`, with the quantizer in `compactbench/presses.py`). Physical bytes are an upper bound on retained information but not an estimate of its entropy, since a byte can be spent on redundant content.

B.4 TASK PROTOCOLS AND PROMPTS

All four tasks use the same needle-in-filler construction. A key-value fact (“The special magic number for {key} is {value}.”) is inserted at a controlled relative depth into a pool of natural

filler text truncated to a target token length. Random seeds are fixed across every budget, so that only the compaction method and its ratio vary between cells.

- **Frontier** (§4.2, Fig. 3): the question “What is the special magic number for $\{key\}$? Answer with just the number.” is asked after compaction, and an answer counts as correct if it contains the value. The reference sweep covers lengths $\{2000, 4000, 8000\}$ tokens $\times$ depths $\{0.1, 0.3, 0.5, 0.7, 0.9\} \times 3$ trials (45 needles per cell) $\times$ six methods (SnapKV, StreamingLLM, TOVA, Knorm, ExpectedAttn, Random) $\times$ ratios $\{0.1, 0.25, 0.5, 0.75, 0.9\}$, plus a full-cache baseline, giving the 1,395 generations behind Figure 3.
- **Reversibility** (§4.4, Table 2): this task does not use the needle. Each document has 24 chunks of repeated synthetic filler (about 1,600 tokens in total), with twelve facts of the form “the $\{attribute\}$ of $\{entity\}$ is $\{value\}$ ” placed in randomly chosen chunks. Both arms read the document chunk by chunk and compact on overflow. The summary arm asks the model to summarize its buffer within the budget while keeping every name and number. The raw arm evicts the oldest chunks and, at answer time, passes on the kept chunks that mention the entity asked about (all kept chunks if none do). Each fact is asked as “What is the $\{attribute\}$ of $\{entity\}$? Answer with just the value.” Budgets are $\{0.1, 0.25, 0.5, 0.75, 1.0\}$ of the document length, with 8 documents (96 questions per budget). The summary may use the whole budget. Generation is capped at the budget plus 40 tokens and the output is truncated to the budget. An earlier version of the harness capped generation at 380 tokens, which cut off any summary longer than that, possible at budgets above 24% of the document. Every reversibility number in the main text uses the corrected harness except those for Qwen3-8B and Phi-4. Rerunning the other 16-bit settings with the correction changed summary accuracy by at most 0.19 and never changed which arm leads at any budget, and under the earlier harness the summary already leads Qwen3-8B’s and Phi-4’s raw buffer by at least 0.21 at every budget below the full history.
- **Frontier variants** (§4.2): the question-aware variant appends “Question: $\{question\}$ ” to the context before compression and asks the question again afterwards. The multikey variant inserts three further needles of the same template for other keys at random depths at least 0.08 apart. Both use the replication subset of Table 5.
- **Mechanisms** (§4.1): the 48 needles of the replication subset (lengths $\{2000, 4000\}$, depths $\{0.1, 0.5, 0.9\}$, 8 trials) are answered after six eviction presses at keep fractions $\{0.5, 0.25, 0.125\}$, KV quantization at $\{8, 4, 3, 2\}$ bits, SnapKV, StreamingLLM, Knorm, and Random eviction followed by quantization at (8 bits, keep 0.5), (4 bits, keep 0.5), and (4 bits, keep 0.25), LLMingua-2 (xlm-roberta-large-meetingbank) at rates $\{0.5, 0.25, 0.125\}$, and a summary written by the model itself, prompted “Summarize the following text in at most $\lfloor 0.75m \rfloor$ words, preserving every specific name and number” and truncated to m tokens. For prompt compression and summaries the BPT is the measured ratio of kept to original tokens.
- **Attribution and calibration** (§4.5): from one compressed cache, the probe “Do your notes contain the special magic number for $\{key\}$? Answer with just yes or no.” and the answer question followed by “Then, on a new line, state how confident you are that your answer is correct as a number from 0 to 100.” are asked in turn, each starting from the same compressed cache, with up to 100 new tokens. Confidence is the last number in $[0, 100]$ in the completion, with the answer’s own digits removed first; a completion without one is recorded as missing and left out of the confidence statistics, not scored as 50. Each of the 48 needles (lengths $\{2000, 4000\}$, depths $\{0.2, 0.5, 0.8\}$, 8 trials) is run under the present and absent controls and under SnapKV, StreamingLLM, Knorm, and Random at ratios $\{0.5, 0.9\}$. Survival is the share of (layer, KV head) slots whose retained set contains each needle token, averaged over the tokens of the key or of the value, computed from the indices the press keeps. Because both questions read one cache, the random press takes part here, which it could not in a two-call design.
- **Dense reversibility** (§4.4): 24 chunks, each with one filler sentence and two facts, 48 facts in about 1,300 tokens, 4 documents (192 questions per budget).

Question-agnostic compression. KVPress compresses the context during prefill and appends the question afterwards, so every method commits to a retained set before it sees the question, as when

a compressed cache is reused across queries. SnapKV’s observation window therefore covers the last 64 tokens of the context, which in the question-agnostic setting is filler, and the question-aware variant of Section 4.2 measures what that costs.

B.5 RUNTIME

A full four-task sweep at the reference scale (the 1.5B model with all the lengths, positions, methods, and ratios above) takes under two hours on a single RTX 4060. Most of the time goes to the attribution and calibration tasks, which need two generations per cell (probe and answer, or answer and confidence). The replication subset used in Section 4.3 is a smaller sweep over fewer lengths, positions, and ratios. It took 23 minutes at 0.5B, 46 at 1.5B fp16, 58 at 1.5B NF4, 73 at 3B, 103 at Phi-3.5-mini, and 102 at 7B, for a total of 6h45m on the same GPU. The quantized settings ran more slowly than their parameter counts alone would suggest, because of NF4 dequantization overhead and longer generations. The third batch ran 44 jobs in 19.1 hours on the same GPU, four of them audits repeated after the confidence parser was fixed: 1.8 hours of frontiers, 4.2 of question-aware and multikey frontiers, 3.3 of audits, 7.5 of standard and dense reversibility, and 2.3 of mechanism sweeps, most of the last two spent writing summaries. Qwen2.5-14B took 42, 23, and 44 minutes for its frontier, audit, and reversibility runs on one RTX 5000 Ada. Per-run logs and raw JSON records (one row per generation, from which the summary statistics in Section 4 are computed as simple aggregates) are released with the benchmark.